

GAME PROGRAMMING PATTERNS

オブジェクトプール & コマンドパターン

全2コマ（各50分） | ゲームプログラミングのデザインパターン入門

対象：3年生

参考資料について

- この授業は、Unity 公式の eBook 「ゲームプログラミングのパターンを活用してコードをレベルアップさせる」を参考にしています
- サンプルプロジェクト（GitHub）にも同じパターンの実装例が収録されています
- コード例はすべて授業用に書き直したオリジナル版です。実際のプロジェクトのコードとは変数名などが異なります



参考 eBook : Unity3D.jp

<https://unity3d.jp/game/game-ebooks/level-up-your-code-with-game-programming-patterns/>



サンプルプロジェクト: GitHub

<https://github.com/Unity-Technologies/game-programming-patterns-demo/>

LESSON 1 | 50 分

オブジェクトプール

大量のオブジェクト生成・破棄を最適化する

1

このコマの目標

- 大量のオブジェクトを生成・破棄することの問題点を理解する
- オブジェクトプールの仕組みを理解する
- Unity 組み込みの `UnityEngine.Pool` を使えるようになる
- サンプルプロジェクトで実際の動きを確認する

悪い例：毎回 Instantiate / Destroy する

- 銃から弾を連射するとき、撃つたびに新しく生成し、使い終わったら破棄する...という素朴な実装です。

```
c# Program.cs
1 // NG : 撃つたびに生成し、使い終わったら破棄している
2 public class NaiveGun : MonoBehaviour
3 {
4     [SerializeField] private Bullet bulletPrefab;
5     public void Fire()
6     {
7         Bullet bullet = Instantiate(
8             bulletPrefab, transform.position, transform.rotation);
9         Destroy(bullet.gameObject, 2f);
10    }
11 }
```

注意

連射するたびに生成・破棄が繰り返され、GC（ガベージコレクション）の負荷が急増する。

- GC の処理が重なるタイミングで、一瞬「カクッ」というスタッター（引っかかり）が起きることがある

オブジェクトプールとは

- 使い終わったオブジェクトを破棄せず、非表示にして「プール」に戻し、再利用する最適化手法
- オブジェクトが必要になったら、新しく作らずプールから取り出して有効化するだけ
- 使い終わったら、破棄する代わりに無効化してプールに返す
 - ー 例え：貸し出し自転車のシェアサービス。使うたびに新しい自転車を作らず、返却されたものを再利用する

自作するとしたら（考え方）

- 取り出す時：プールが空でなければ取り出して有効化。空なら新規作成する。返す時：無効化してプールに戻す。

c# Program.cs

```
1 public class BulletPool : MonoBehaviour
2 {
3     [SerializeField] private Bullet bulletPrefab;
4     private Stack<Bullet> pool = new Stack<Bullet>();
5     public Bullet Get()
6     {
7         if (pool.Count == 0)
8         {
9             return Instantiate(bulletPrefab);
10        }
11        Bullet next = pool.Pop();
12        next.gameObject.SetActive(true);
13        return next;
14    }
15    public void Return(Bullet bullet)
16    {
17        bullet.gameObject.SetActive(false);
18        pool.Push(bullet);
19    }
20 }
```

Unity 組み込みの UnityEngine.Pool

- 「作る／取得する／返す／破棄する」タイミングの処理をそれぞれ登録するだけで、自作の Stack 管理が不要になる。

C# Program.cs

```
1 using UnityEngine.Pool;
2 public class BulletSpawner : MonoBehaviour
3 {
4     [SerializeField] private Bullet bulletPrefab;
5     private IObjectPool<Bullet> bulletPool;
6     void Awake()
7     {
8         bulletPool = new ObjectPool<Bullet>(
9             CreateBullet, OnGet, OnReturn, OnDestroyBullet);
10    }
11    Bullet CreateBullet() => Instantiate(bulletPrefab);
12    void OnGet(Bullet b) => b.gameObject.SetActive(true);
13    void OnReturn(Bullet b) => b.gameObject.SetActive(false);
14    void OnDestroyBullet(Bullet b) => Destroy(b.gameObject);
15 }
```


RevisedGun.cs の流れ（目次）

- ① 使う名前空間とフィールド
- ② プール関連のフィールド
- ③ Awake()
- ④ CreateProjectile()
- ⑤ OnReleaseToPool()
- ⑥ OnGetFromPool()
- ⑦ OnDestroyPooledObject()
- ⑧ FixedUpdate()
- ⑨ Shoot() 前半
- ⑩ Shoot() 後半

POINT

この後、①から順番に1つずつコードと処理内容を見ていきます。

RevisedGun.cs ①：使う名前空間とフィールド

c# Program.cs

```
1 using UnityEngine;
2 using UnityEngine.Pool;
3 using UnityEngine.Events;
4 public class RevisedGun : MonoBehaviour
5 {
6     [Tooltip("Prefab to shoot")]
7     [SerializeField] private RevisedProjectile projectilePrefab;
8     [Tooltip("Projectile force")]
9     [SerializeField] private float muzzleVelocity = 1500f;
10    [Tooltip("End point of gun where shots appear")]
11    [SerializeField] private Transform muzzlePosition;
12    [Tooltip("Time between shots / smaller = higher rate of fire")]
13    [SerializeField] private float cooldownWindow = 0.1f;
```

- using UnityEngine.Pool; を書くことで、組み込みのオブジェクトプール機能が使えるようになる
- [Tooltip("...")] はインスペクターにヒント文を表示するための属性で、処理には影響しない
- projectilePrefab（弾の元）、muzzleVelocity（弾の初速）、muzzlePosition（発射位置）、cooldownWindow（連射間隔）の4つを最初に定義している

RevisedGun.cs ②：プール関連のフィールド

C# Program.cs

```
1  [SerializeField] private UnityEvent m_GunFired;  
2  
3  // Stack-based ObjectPool available with Unity 2021 and above  
4  private IObjectPool<RevisedProjectile> objectPool;  
5  
6  // Throw an exception if we try to return an existing item, already in the pool  
7  [SerializeField] private bool collectionCheck = true;  
8  
9  // extra options to control the pool capacity and maximum size  
10 [SerializeField] private int defaultCapacity = 20;  
11 [SerializeField] private int maxSize = 100;  
12  
13 private float nextTimeToShoot;
```

- `m_GunFired`：発射のたびに呼ばれるイベント（音やエフェクトなどを鳴らすのに使える）
- `objectPool`： `IObjectPool<T>` 型。私たちが自作したときの `Stack<T>` の役割をこれ 1 つが担う
- `collectionCheck`： `true` だと、すでにプールに戻したオブジェクトを二重に返そうとしたときにエラーで教えてくれる
- `defaultCapacity` / `maxSize`：プールの初期サイズと上限サイズ。`nextTimeToShoot` は連射間隔の管理用

RevisedGun.cs ③ : Awake()

c# Program.cs

```
1 private void Awake()  
2 {  
3     objectPool = new ObjectPool<RevisedProjectile>(CreateProjectile,  
4         OnGetFromPool, OnReleaseToPool, OnDestroyPooledObject,  
5         collectionCheck, defaultCapacity, maxSize);  
6 }
```

- Awake は、このスクリプトが読み込まれた直後（ゲーム開始前）に 1 回だけ呼ばれる Unity のメソッド
- new ObjectPool<T>(...) の引数には、「作る」「取得時」「返却時」「破棄時」の 4 つのメソッドを順番に渡す
- 続けて collectionCheck, defaultCapacity, maxSize を渡すことで、プールの安全設定とサイズを指定している
- この 1 回のコンストラクタ呼び出しだけで、プールの仕組み全体が組み上がる

RevisedGun.cs ④ : CreateProjectile()

c# Program.cs

```
1 // invoked when creating an item to populate the object pool
2 private RevisedProjectile CreateProjectile()
3 {
4     RevisedProjectile projectileInstance = Instantiate(projectilePrefab);
5     projectileInstance.ObjectPool = objectPool;
6     return projectileInstance;
7 }
```

- プールの中身が足りないときに「新しく 1 つ作る」役割のメソッド
- Instantiate(projectilePrefab) で弾のプレハブを実体化する
- projectileInstance.ObjectPool = objectPool; で、作った弾自身に「自分がどのプールに属しているか」を覚えておく
- → 弾が自分で objectPool.Release(this) を呼んでプールに戻れるようにするため

RevisedGun.cs ⑤ : OnReleaseToPool()

c# Program.cs

```
1 // Invoked when returning an item to the object pool
2 private void OnReleaseToPool(RevisedProjectile pooledObject)
3 {
4     pooledObject.gameObject.SetActive(false);
5 }
```

- 弾がプールに「返却」されるタイミングで呼ばれるメソッド
- SetActive(false) でゲームオブジェクトを非表示・非アクティブにするだけのシンプルな処理
- 破棄（Destroy）はしていない点に注目。あくまで「隠して待機させる」だけ

RevisedGun.cs ⑥ : OnGetFromPool()

c# Program.cs

```
1 // Invoked when retrieving the next item from the object pool
2 private void OnGetFromPool(RevisedProjectile pooledObject)
3 {
4     pooledObject.gameObject.SetActive(true);
5 }
```

- プールから弾を「取得」するタイミングで呼ばれるメソッド
- OnReleaseToPool のちょうど逆で、SetActive(true) にして再び表示・動作させる
- この2つのメソッドのペアが、オブジェクトプールの「隠す／出す」を担っている

RevisedGun.cs ⑦ : OnDestroyPooledObject()

c# Program.cs

```
1 // Invoked when we exceed the maximum number of pooled items
2 // (i.e. destroy the pooled object)
3 private void OnDestroyPooledObject(RevisedProjectile pooledObject)
4 {
5     Destroy(pooledObject.gameObject);
6 }
```

- プールの上限（maxSize）を超えたときだけ呼ばれるメソッド
- ここでは非表示にするのではなく、実際に Destroy() で破棄している
- 「上限を超えた分は捨てる」という設定により、メモリの使いすぎを防いでいる

RevisedGun.cs ⑧ : FixedUpdate()

c# Program.cs

```
1 private void FixedUpdate()
2 {
3     // Shoot if we have exceeded delay
4     if (Input.GetButton("Fire1") && Time.time > nextTimeToShoot
5         && objectPool != null)
6     {
7         Shoot();
8     }
9 }
```

- 毎フレーム呼ばれ、発射ボタンが押されているかを確認するメソッド
- 3つの条件（発射ボタンが押されている／クールダウンが過ぎている／プールが準備済み）が揃ったときだけ Shoot() を呼ぶ
- この条件チェックのおかげで、ボタンを押しっぱなしにしても連射間隔が守られる

RevisedGun.cs ⑨ : Shoot() 前半

C# Program.cs

```
1 private void Shoot()
2 {
3     // Get a pooled object instead of instantiating
4     RevisedProjectile bulletObject = objectPool.Get();
5
6     if (bulletObject == null)
7         return;
8
9     // Align to gun barrel/muzzle position
10    bulletObject.transform.SetPositionAndRotation(
11        muzzlePosition.position, muzzlePosition.rotation);
```

- `objectPool.Get()` が、このスクリプトの中で一番大事な 1 行
- `Instantiate` ではなく `Get` を呼ぶことで、プール内の待機中オブジェクトが再利用される（足りなければ内部で `CreateProjectile` が呼ばれる）
- 取得した弾の位置と向きを、銃口（`muzzlePosition`）に合わせている

RevisedGun.cs ⑩ : Shoot() 後半

C# Program.cs

```
1      // Move projectile forward
2      bulletObject.GetComponent<Rigidbody>().AddForce(
3          bulletObject.transform.forward * muzzleVelocity,
4          ForceMode.Acceleration);
5
6      // Turn off after a few seconds
7      bulletObject.Deactivate();
8
9      // Set cooldown delay
10     nextTimeToShoot = Time.time + cooldownWindow;
11
12     m_GunFired.Invoke();
13 }
14 }
```

- Rigidbody に AddForce で力を加え、弾を前方に飛ばす
- Deactivate() は弾（Projectile）側のスクリプトの処理で、一定時間後に objectPool.Release(this) を呼んでプールに戻す
- 最後に nextTimeToShoot を更新し、m_GunFired（発射イベント）を呼んで処理が完了する

長所と短所

長所

- GC スパイクを抑え、なめらかな動作を保てる
- 大量の弾やエフェクトを連射しても軽い
- 組み込み API を使えば実装もシンプル

短所

- 管理するコレクションが増え、構造がやや複雑になる
- プールのサイズ設定を誤ると、メモリを無駄に使うことがある
- 小規模でオブジェクト数が少ない場面では過剰な最適化になることも

LESSON 2 | 50 分

コマンドパターン

「操作」をオブジェクトとして扱い、取り消し・やり直しを実現する

2

このコマの目標

- 「操作」をオブジェクトとして扱う考え方を理解する
- Undo（取り消し）・Redo（やり直し）の仕組みを理解する
- コマンドパターンを使った移動処理を実装できるようになる
- サンプルプロジェクトで実際の動きを確認する

悪い例：直接メソッドを呼び出す

- ボタンを押したら、プレイヤーを直接動かすだけの素朴な実装です。取り消し（Undo）ができません。

C# Program.cs

```
1 // NG : 直接 Move を呼ぶだけで、元に戻す手段がない
2 public class NaiveInputManager : MonoBehaviour
3 {
4     [SerializeField] private PlayerMover player;
5     public void OnUpButton()
6     {
7         player.Move(Vector3.forward);
8     }
9 }
```

注意

「何をしたか」がどこにも残らないため、あとから取り消す（Undo）方法がない。

- → 操作そのものを「オブジェクト」として保存できないか？

コマンドパターンとは

- 「実行したい処理」を、メソッド呼び出しの代わりにオブジェクトとしてカプセル化するパターン
- ICommand インターフェイスに Execute（実行）と Undo（取り消し）を持たせる
- コマンドをスタックなどのコレクションに積んでおけば、後から取り消し・再生ができる

c# Program.cs

```
1 public interface ICommand
2 {
3     void Execute();
4     void Undo();
5 }
```


コマンドを管理する仕組み

- 取り消しスタックとやり直しスタックの、2つのスタックで履歴を管理する。

c# Program.cs

```
1 public class CommandInvoker
2 {
3     private Stack<ICommand> undoStack = new Stack<ICommand>();
4     private Stack<ICommand> redoStack = new Stack<ICommand>();
5     public void ExecuteCommand(ICommand command)
6     {
7         command.Execute();
8         undoStack.Push(command);
9         redoStack.Clear();
10    }
11    public void Undo()
12    {
13        if (undoStack.Count == 0) return;
14        ICommand command = undoStack.Pop();
15        command.Undo();
16        redoStack.Push(command);
17    }
18 }
```

CommandInvoker.cs の流れ（目次）

- ① using 文とクラス宣言
- ② s_UndoStack フィールド
- ③ s_RedoStack フィールド
- ④ ExecuteCommand() 前半
- ⑤ ExecuteCommand() 後半
- ⑥ UndoCommand() 前半
- ⑦ UndoCommand() 後半
- ⑧ RedoCommand() 前半
- ⑨ RedoCommand() 後半
- ⑩ 全体の流れ

POINT

この後、①から順番に1つずつコードと処理内容を見ていきます。

CommandInvoker.cs ① : using 文とクラス宣言

c# Program.cs

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class CommandInvoker
6 {
```

- `using System.Collections.Generic;` を書くことで、この後に出てくる `Stack<T>` が使えるようになる
- `CommandInvoker` という、コマンドの実行・取り消し・やり直しをすべて担当するクラスを定義していく

CommandInvoker.cs ② : s_UndoStack フィールド

c# Program.cs

```
1 // Stack of command objects to undo
2 private static Stack<ICommand> s_UndoStack = new Stack<ICommand>();
```

- 取り消し（Undo）待ちのコマンドを積んでおくスタック
- static なので、このクラス自体は new せずに使う（インスタンスが1つに定まる＝シングルトンに近い考え方）
- 先頭の s_ は「static」であることを示す、このプロジェクトの命名ルール（プレフィックス）
- Stack（スタック）は「後入れ先出し」：最後に積んだコマンドから先に取り消される

CommandInvoker.cs ③ : s_RedoStack フィールド

c# Program.cs

```
1 // Second stack of redoable commands
2 private static Stack<ICommand> s_RedoStack = new Stack<ICommand>();
```

- やり直し（Redo）待ちのコマンドを積んでおく、もう 1 つのスタック
- 取り消したコマンドは消えるのではなく、いったんこちらに移される
- スタックが 2 つあることで、「取り消し」と「やり直し」の両方に対応できる

CommandInvoker.cs ④ : ExecuteCommand() 前半

c# Program.cs

```
1 // Execute a command object directly and save to the undo stack
2 public static void ExecuteCommand(ICommand command)
3 {
4     command.Execute();
```

- コマンドを実行するときの入口になるメソッド
- 引数で受け取った command (ICommand を実装したオブジェクト) の Execute() を、ここで実際に呼び出している
- 呼び出す側 (ButtonInputs など) は、中身のロジックを知らなくても command.Execute() の 1 行だけで処理を任せられる

CommandInvoker.cs ⑤ : ExecuteCommand() 後半

c# Program.cs

```
1      s_UndoStack.Push(command);  
2  
3      // Clear out the redo stack if we make a new move  
4      s_RedoStack.Clear();  
5  }
```

- 実行したコマンドを s_UndoStack に積んでおく。あとで取り消せるようにするため
- 同時に s_RedoStack を空にしている点が重要
- 新しい操作をした時点で、それより前の「やり直し」履歴は無効になる（一般的な Undo/Redo の動作と同じ）

CommandInvoker.cs ⑥ : UndoCommand() 前半

c# Program.cs

```
1 public static void UndoCommand()  
2 {  
3     // If we have commands to undo  
4     if (s_UndoStack.Count > 0)  
5     {
```

- 取り消しを行うメソッド
- `s_UndoStack.Count > 0` で「取り消せるコマンドが残っているか」を確認している
- スタックが空のときは何もしない（エラーにならないようにするための安全チェック）

CommandInvoker.cs ⑦ : UndoCommand() 後半

c# Program.cs

```
1      ICommand activeCommand = s_UndoStack.Pop();  
2      s_RedoStack.Push(activeCommand);  
3      activeCommand.Undo();  
4  }  
5 }
```

- `s_UndoStack.Pop()` で直近のコマンドを取り出す（スタックから取り除かれる）
- 取り出したコマンドを `s_RedoStack` に積み直しておく＝あとで「やり直し」できるようにする
- 最後に `activeCommand.Undo()` を呼んで、実際に元の状態に戻す処理を実行する

CommandInvoker.cs ⑧ : RedoCommand() 前半

c# Program.cs

```
1 public static void RedoCommand()  
2 {  
3     // If we have commands to redo  
4     if (s_RedoStack.Count > 0)  
5     {
```

- やり直しを行うメソッド。UndoCommand とほぼ対になる構造
- s_RedoStack.Count > 0 で「やり直せるコマンドが残っているか」を確認する

CommandInvoker.cs ⑨ : RedoCommand() 後半

```
c# Program.cs
1      ICommand activeCommand = s_RedoStack.Pop();
2      s_UndoStack.Push(activeCommand);
3      activeCommand.Execute();
4  }
5  }
6  }
```

- s_RedoStack から取り出したコマンドを、今度は s_UndoStack に戻す
- Undo のときと逆で、最後は activeCommand.Execute() を呼んで操作をもう一度実行する
- これで「取り消したものを、もう一度やり直す」動きが完成する

CommandInvoker.cs ⑩ : 全体の流れ

- ① ExecuteCommand : コマンドを実行し、UndoStack に積む (RedoStack は空にする)
- ② UndoCommand : UndoStack から取り出して取り消し、RedoStack に積む
- ③ RedoCommand : RedoStack から取り出してもう一度実行し、UndoStack に積む
- 2つのスタックの間をコマンドが行き来することで、Undo ・ Redo の両方が実現されている
- 呼び出す側は、ExecuteCommand / UndoCommand / RedoCommand の 3 つを呼ぶだけで良く、内部のスタック操作を意識する必要がない

例：移動コマンド

- 「対象を動かす」処理を MoveCommand というオブジェクトにする。Execute で移動、Undo で逆方向に動かせば元に戻る。

C# Program.cs

```
1 public class MoveCommand : ICommand
2 {
3     private IMovable target;
4     private Vector3 direction;
5     public MoveCommand(IMovable target, Vector3 direction)
6     {
7         this.target = target;
8         this.direction = direction;
9     }
10    public void Execute()
11    {
12        target.Move(direction);
13    }
14    public void Undo()
15    {
16        target.Move(-direction);
17    }
18 }
```

ButtonInputs.cs の流れ（目次）

- ① using 文とキー設定
- ② ボタンとプレイヤーの参照
- ③ Start() でボタンを登録
- ④ RunPlayerCommand() 前半
- ⑤ RunPlayerCommand() 後半
- ⑥ Update() 前半（移動キー）
- ⑦ Update() 中盤（左右キー）
- ⑧ Update() 後半（Undo/Redo キー）
- ⑨ 移動系の On ~ Input()
- ⑩ Undo ・ Redo の On ~ Input()

POINT

この後、①から順番に1つずつコードと処理内容を見ていきます。

ButtonInputs.cs ① : using 文とキー設定

c# Program.cs

```
1 using System;
2 using System.Collections;
3 using System.Collections.Generic;
4 using UnityEngine;
5 using UnityEngine.UI;
6
7 public class ButtonInputs : MonoBehaviour
8 {
9     [Header("Key Controls")]
10    [SerializeField] private KeyCode m_ForwardKey = KeyCode.W;
11    [SerializeField] private KeyCode m_BackKey = KeyCode.S;
12    [SerializeField] private KeyCode m_LeftKey = KeyCode.A;
13    [SerializeField] private KeyCode m_RightKey = KeyCode.D;
14    [SerializeField] private KeyCode m_UndoKey = KeyCode.U;
15    [SerializeField] private KeyCode m_RedoKey = KeyCode.R;
```

- using UnityEngine.UI; を追加することで、Button などの UI 部品が使えるようになる
- [Header("Key Controls")] はインスペクター上に見出しを表示する属性
- 移動 4 方向（W/A/S/D）と Undo（U）・Redo（R）、合計 6 つのキーを変数として持たせている

ButtonInputs.cs ② : ボタンとプレイヤーの参照

C# Program.cs

```
1 // UI Button controls
2 [Header("Button Controls")]
3 [SerializeField] private Button m_ForwardButton;
4 [SerializeField] private Button m_BackButton;
5 [SerializeField] private Button m_LeftButton;
6 [SerializeField] private Button m_RightButton;
7 [SerializeField] private Button m_UndoButton;
8 [SerializeField] private Button m_RedoButton;
9
10 [SerializeField] private PlayerMover m_Player;
```

- キーボードと同じ 6 つの操作に対応する、画面上の UI ボタンをそれぞれ変数として持たせている
- m_Player : 実際に動かす対象 (PlayerMover) への参照。コマンドの宛先になる

ButtonInputs.cs ③ : Start() でボタンを登録

C# Program.cs

```
1  private void Start()
2  {
3      // button setup
4      m_ForwardButton.onClick.AddListener(OnForwardInput);
5      m_BackButton.onClick.AddListener(OnBackInput);
6      m_RightButton.onClick.AddListener(OnRightInput);
7      m_LeftButton.onClick.AddListener(OnLeftInput);
8      m_UndoButton.onClick.AddListener(OnUndoInput);
9      m_RedoButton.onClick.AddListener(OnRedoInput);
10 }
```

- Start は、ゲーム開始時に 1 回だけ呼ばれる Unity のメソッド
- AddListener で「このボタンが押されたら、このメソッドを呼ぶ」という対応づけを行っている
- 6 つのボタンすべてに、対応する On ~ Input メソッドを登録している

ButtonInputs.cs ④ : RunPlayerCommand() 前半

```
c# Program.cs
1 private void RunPlayerCommand(PlayerMover playerMover, Vector3 movement)
2 {
3     if (playerMover == null)
4     {
5         return;
6     }
```

- 移動コマンドをまとめて発行するための、共通の入口メソッド
- 最初に playerMover が null でないかを確認し、null なら何もせず抜ける（安全チェック）

ButtonInputs.cs ⑤ : RunPlayerCommand() 後半

C# Program.cs

```
1      // check if movement is unobstructed
2      if (playerMover.IsValidMove(movement))
3      {
4          // issue the command and save to undo stack
5          ICommand command = new MoveCommand(playerMover, movement);
6
7          // we run the command immediately here, but you can also
8          // delay this for extra control over the timing
9          CommandInvoker.ExecuteCommand(command);
10     }
11 }
```

- IsValidMove で「壁にぶつからないか」を確認してから動かす
- new MoveCommand(...) でコマンドオブジェクトを作り、CommandInvoker.ExecuteCommand に渡すだけ
- PlayerMover を直接動かすのではなく、必ずコマンド経由にすることで Undo/Redo に対応できる

ButtonInputs.cs ⑥ : Update() 前半 (移動キー)

c# Program.cs

```
1  private void Update()
2  {
3      // Check for forward movement key
4      if (Input.GetKeyDown(m_ForwardKey))
5      {
6          m_ForwardButton.onClick.Invoke();
7      }
8
9      // Check for back movement key
10     if (Input.GetKeyDown(m_BackKey))
11     {
12         m_BackButton.onClick.Invoke();
13     }
```

- Update は毎フレーム呼ばれ、キーボード入力を監視している
- GetKeyDown でキーが押された瞬間を検知し、対応するボタンの onClick.Invoke() を呼ぶ
- → キーボードで操作しても、ボタンを押したのと同じ処理 (RunPlayerCommand 経由) が実行される

ButtonInputs.cs ⑦ : Update() 中盤 (左右キー)

C# Program.cs

```
1      // Check for left movement key
2      if (Input.GetKeyDown(m_LeftKey))
3      {
4          m_LeftButton.onClick.Invoke();
5      }
6
7      // Check for right movement key
8      if (Input.GetKeyDown(m_RightKey))
9      {
10         m_RightButton.onClick.Invoke();
11     }
```

- 前後キーと全く同じパターンを、左右キーにも繰り返している
- 同じ形の if 文が 6 つ並ぶことで、6 つの操作すべてに同じ仕組みが使われていることが分かる

ButtonInputs.cs ⑧ : Update() 後半 (Undo/Redo キー)

C# Program.cs

```
1      // Check for undo key
2      if (Input.GetKeyDown(m_UndoKey))
3      {
4          m_UndoButton.onClick.Invoke();
5      }
6
7      // Check for redo key
8      if (Input.GetKeyDown(m_RedoKey))
9      {
10         m_RedoButton.onClick.Invoke();
11     }
12 }
```

- U キーで取り消し、R キーでやり直しができるようになっている
- 移動キーと同じ仕組みで、ボタンの onClick を呼ぶだけなので処理が重複しない

ButtonInputs.cs ⑨ : 移動系の On ~ Input()

c# Program.cs

```
1  private void OnLeftInput()
2  {
3      RunPlayerCommand(m_Player, Vector3.left);
4  }
5
6  private void OnRightInput()
7  {
8      RunPlayerCommand(m_Player, Vector3.right);
9  }
10
11 private void OnForwardInput()
12 {
13     RunPlayerCommand(m_Player, Vector3.forward);
14 }
15
16 private void OnBackInput()
17 {
18     RunPlayerCommand(m_Player, Vector3.back);
19 }
```

- 4方向それぞれのメソッドは、RunPlayerCommand に方向ベクトルを渡すだけ
- 個々のメソッドは1行しかなく、実際の判断や処理はすべて RunPlayerCommand に任せている

ButtonInputs.cs ⑩ : Undo ・ Redo の On ~ Input()

```
c# Program.cs
1  private void OnUndoInput()
2  {
3      CommandInvoker.UndoCommand();
4  }
5
6  private void OnRedoInput()
7  {
8      CommandInvoker.RedoCommand();
9  }
10 }
```

- 移動系とは違い、RunPlayerCommand を経由せず CommandInvoker を直接呼んでいる
- 取り消し・やり直しは「新しい操作」ではなく、すでに積まれたコマンドを操作するだけだから
- ButtonInputs はどのコマンドがどう動くかを知らず、CommandInvoker に判断を任せている点がポイント

長所と短所

長所

- Undo ・ Redo の実装が、コマンドオブジェクトを貯めるだけで済む
- 入力履歴を使ったコンボやリプレイにも応用できる
- 新しいコマンドを追加しても、既存のコマンドに影響しない

短所

- 操作の種類ごとにクラスが増え、構造がやや複雑になる
- Undo 履歴が増えすぎるとメモリを圧迫するため上限が必要
- 単純な操作しかない場合は、過剰な設計になることもある

お疲れさまでした

オブジェクトプールとコマンドパターンは、
他のデザインパターンと組み合わせることでさらに力を発揮します。